difftree

Differentiable decision trees in JAX with built-in uncertainty

John Kitchin
Carnegie Mellon University · Chemical Engineering

Decision trees: the classic recipe

- Recursively split data on **one feature** at a time
- Choose each split **greedily** (best impurity drop now)
- Predict with a constant in each leaf
- Fast, interpretable, handles mixed features
- CART (Breiman et al., 1984), ID3 (Quinlan, 1986)

A hard, axis-aligned split

```
if x[3] < 0.42: go left else: go right
```

Every input follows exactly **one** root-to-leaf path.

Three things classic trees cannot do

Optimize jointly

Greedy growth fixes early splits; later splits cannot fix them. No gradient flows through `if/else`.

Split obliquely

Axis-aligned cuts need staircases to follow a diagonal or curved boundary.

Say how sure they are

A leaf constant gives a point estimate, not a calibrated predictive distribution.

difftree makes every split a smooth function, so the whole tree trains end to end by gradient descent, and keeps that smoothness at test time to report uncertainty.

Where difftree sits

Greedy trees
CART, ID3

›

Oblique trees
OC1, non-greedy

›

Soft / neural trees
DNDF, soft DT, ANT

›

End-to-end trees
anneal $\gamma \to \infty$

›

difftree
finite γ + UQ

- Hehn et al. (2020) train trees end to end, then **anneal to hard splits**
- difftree keeps splits **finitely soft**: routing ambiguity becomes uncertainty
- Leaves are **Gaussians**, so the output is a calibrated mixture

How it works

A soft, oblique split

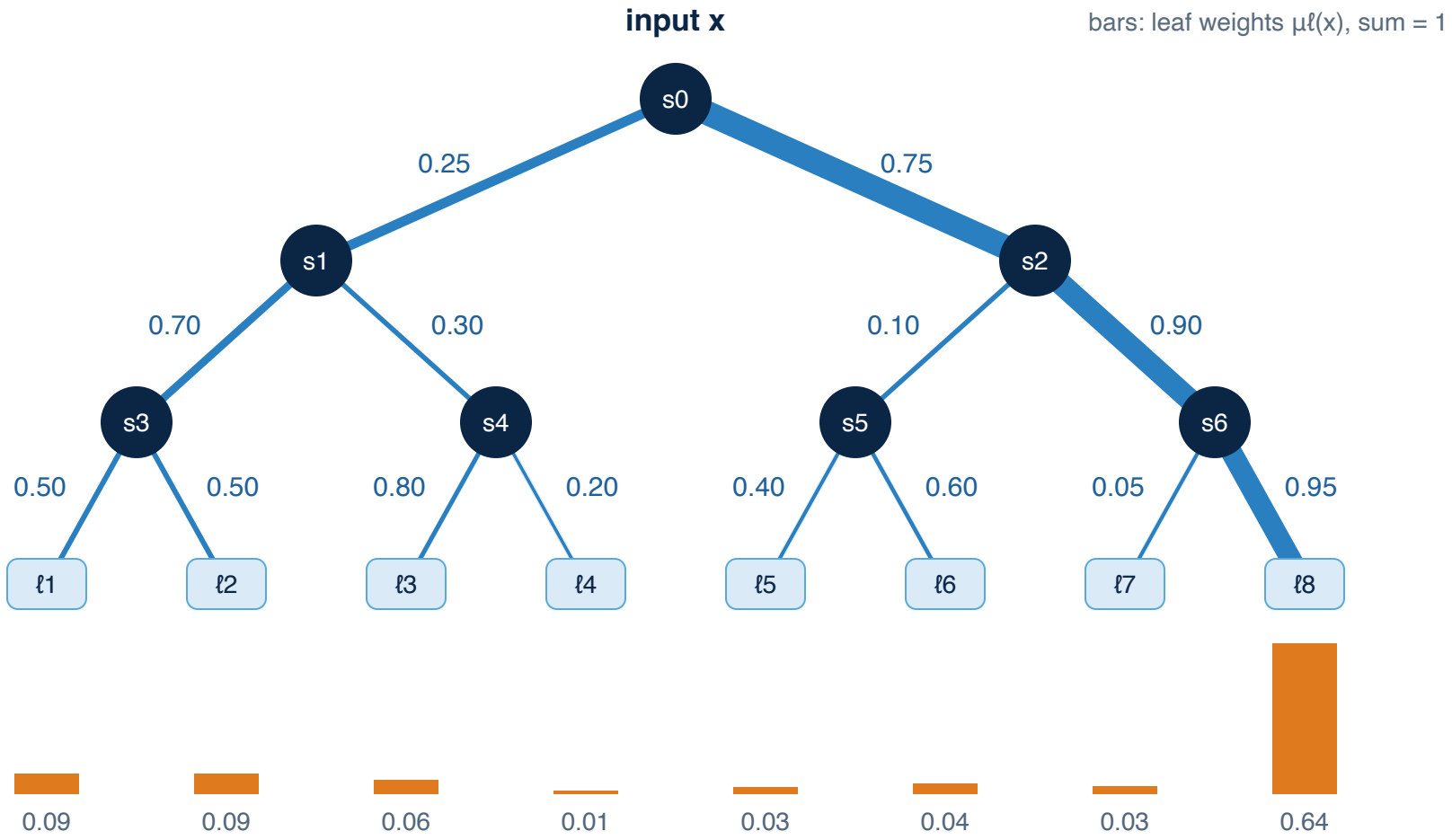
$$s_i(x) = \sigma\big(\gamma_d \, (w_i^\top x + b_i)\big)$$

- w_i: an **oblique** direction (all features at once)
- γ_d: per-depth **steepness**
- s_i = probability of going right
- $\gamma \to \infty$ recovers a hard split

$$\mu_\ell(x) = \prod_{i \in \mathrm{path}(\ell)} s_i^{\,r_i} \, (1 - s_i)^{1 - r_i}$$

Path probabilities over all 2^D leaves sum to one.
Computed in log space for stability.

Soft routing: one input, every leaf



Edge width ∝ probability of reaching that node. Labels are the split probabilities.

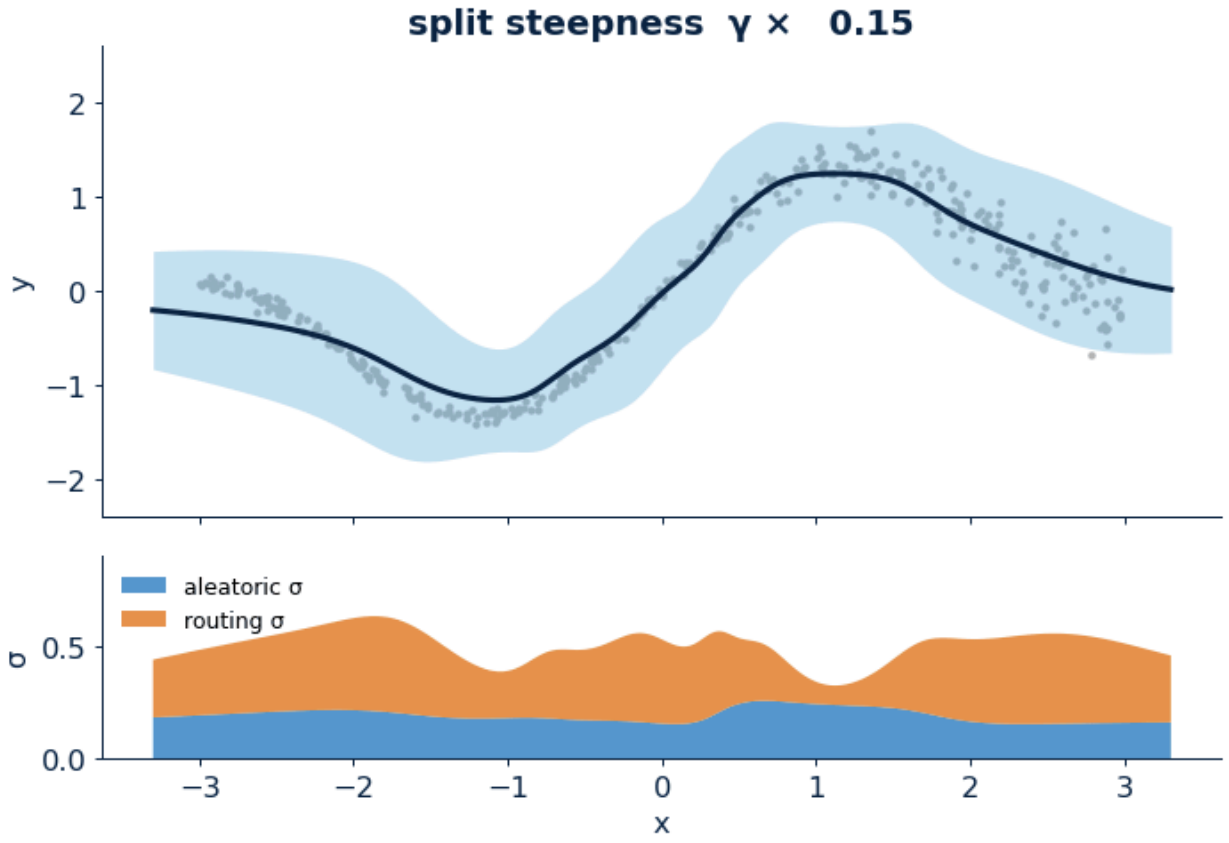
Leaves are Gaussians, so predictions are mixtures

$$p(y \mid x) = \sum_{\ell=1}^{2^D} \mu_\ell(x)\, \mathcal{N}\big(y;\, m_\ell,\, \alpha\, L_\ell L_\ell^\top\big)$$

$$\mathrm{Cov}[y \mid x] = \underbrace{\mathbb{E}_\ell\big[\Sigma_\ell\big]}_{\text{aleatoric (within leaf)}} + \underbrace{\mathrm{Cov}_\ell\big[m_\ell\big]}_{\text{routing (between leaves)}}$$

- Each leaf has its own mean and **Cholesky** covariance (PD by construction)
- Multi-output y gets a full covariance, not just per-output error bars
- Classification: leaves hold logits; entropy splits the same way

Steepness γ controls smoothness and routing uncertainty



- Small γ: smooth mean, large **routing** σ (orange)
- Large γ: piecewise constant, routing σ only at the boundaries
- Calibration picks a **finite** γ per depth
- Aleatoric σ (blue) is the learned noise level

Training: maximize likelihood, then calibrate

Warmup
splits + means, σ frozen

›

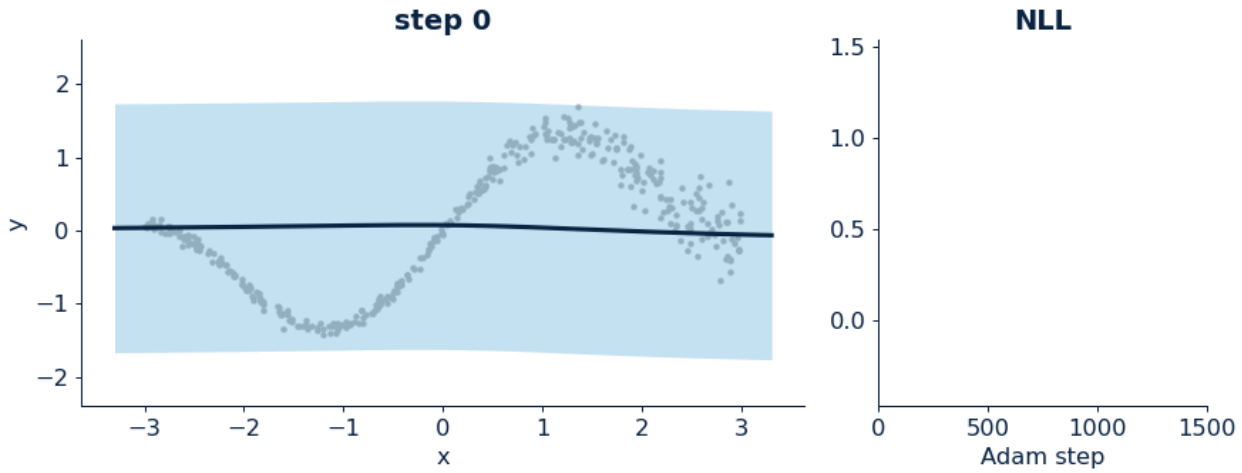
Full NLL
Adam, grad clipping

›

Anneal γ
$1 \rightarrow \gamma_{max}$

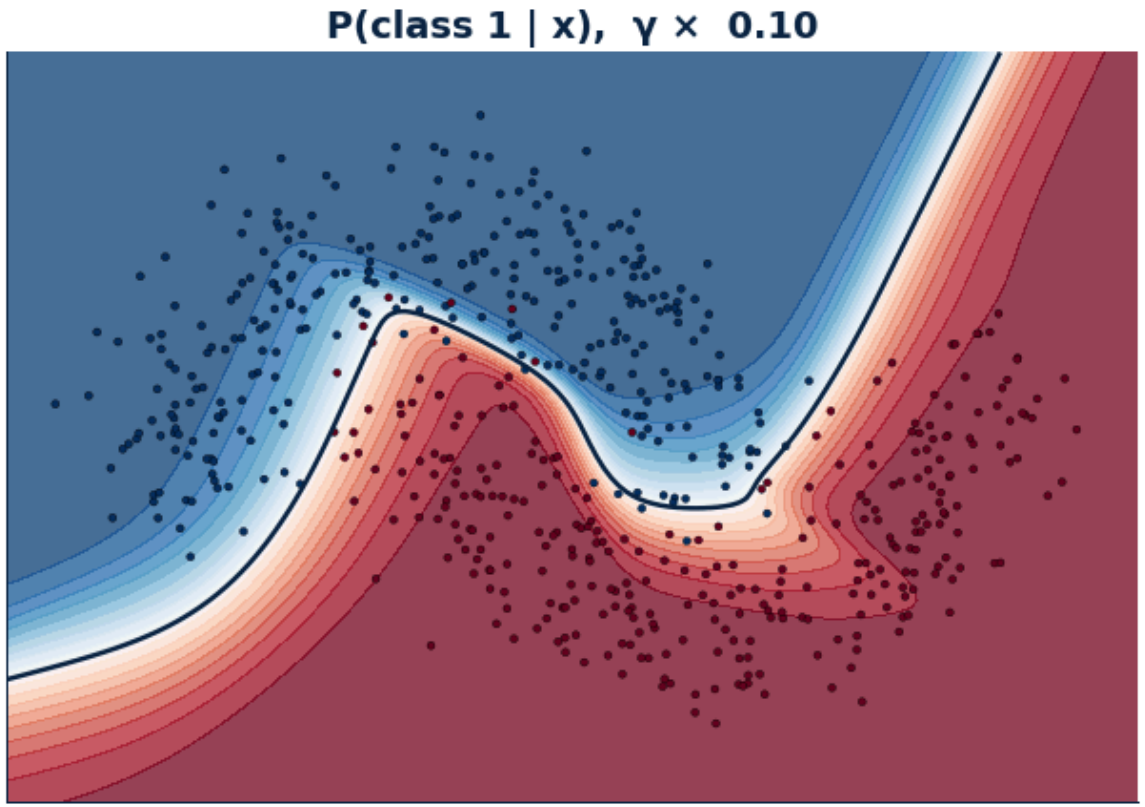
›

Calibrate
fit log γ_d, log α on held-out data



- Loss: $-\log p(y \mid x)$, end to end
- Warmup avoids variance collapse
- Calibration is tree-style **temperature scaling**

Oblique splits bend to the data



- Depth-4 classifier on two moons
- Each cut is a line in **any direction**
- 15 splits trace a curved boundary
- Soft γ: graded probabilities near the boundary
- Hard γ: a polygon partition

Examples

A scikit-learn estimator

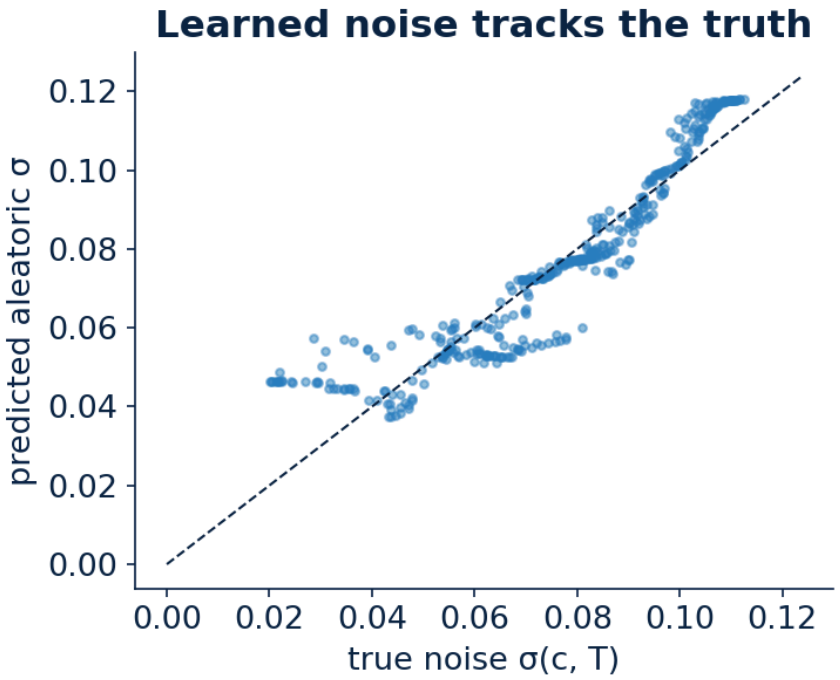
```
from difftree import DifferentiableTreeRegressor

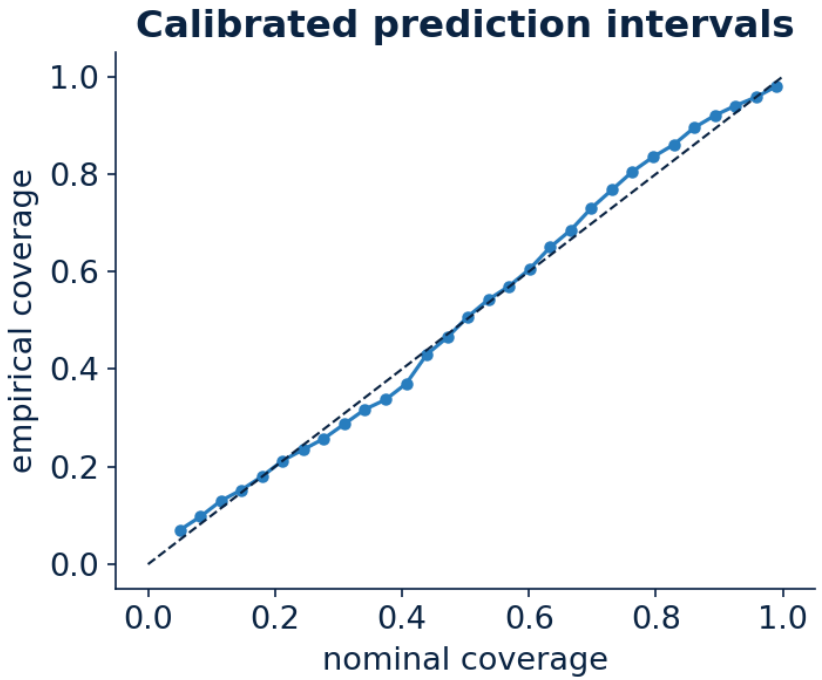
model = DifferentiableTreeRegressor(depth=5, n_epochs=150, random_state=0)
model.fit(X_train, y_train)

mean, std = model.predict(X_test, return_std=True)
mean, std, std_aleatoric, std_routing = model.predict(X_test, return_components=True)
q05, q50, q95 = model.predict_quantile(X_test, q=[0.05, 0.5, 0.95])
samples = model.sample_y(X_test, n_samples=200)
```

Also: `DifferentiableTreeClassifier`, `predict_full_cov`, `predict_conditional`, `inverse_design`. Works with pipelines, CV and grid search.

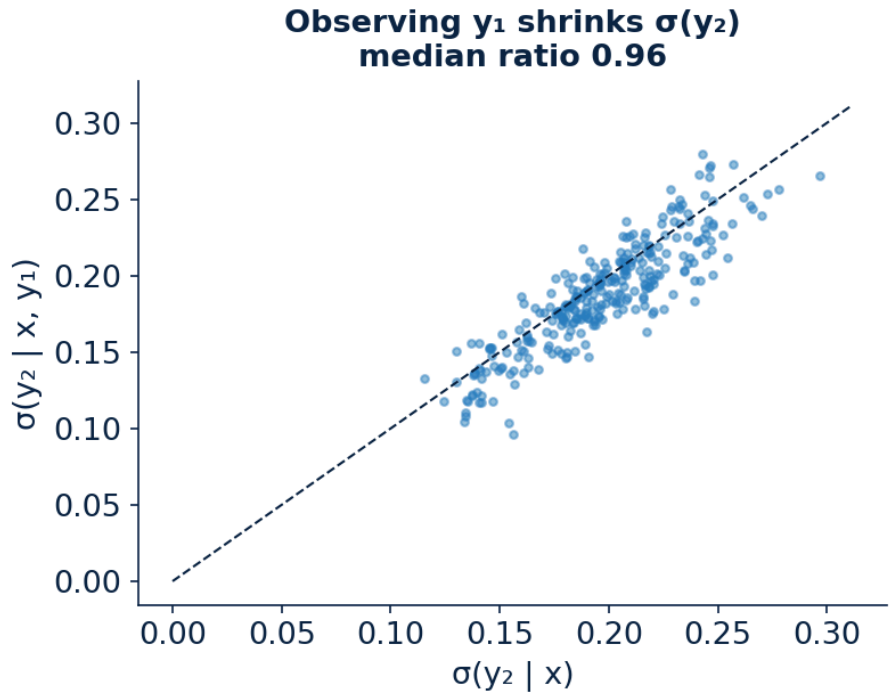
Example 1: a saturating sensor with heteroscedastic noise





Michaelis-Menten response in (c, T) with Arrhenius $K_m(T)$; noise $\sigma = 0.02 + 0.1\, f(c, T)$. Depth 5, 1600 training points. The aleatoric σ recovers the noise structure and intervals are calibrated.

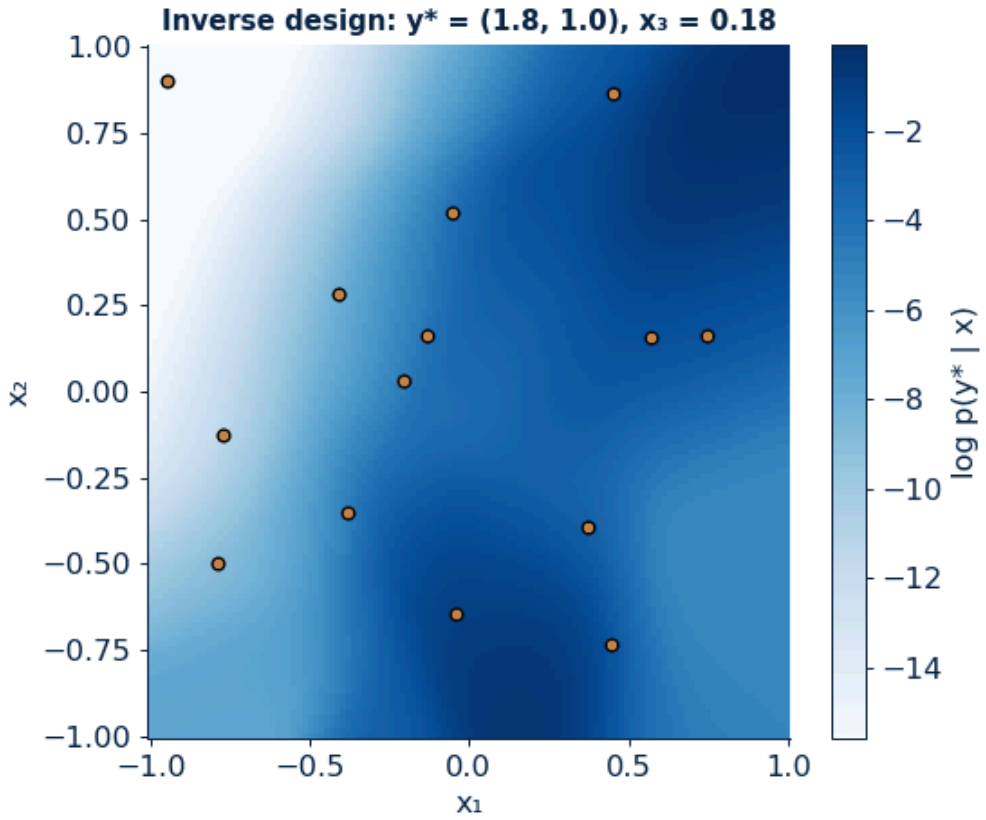
Example 2: knowing one output tightens the other



- Two correlated properties y_1, y_2 from three inputs
- $p(y_2 \mid x, y_1)$ in **closed form**: Schur complement per leaf, leaf weights reweighted by evidence
- Conditional σ is smaller for most inputs, but modestly here (median ratio 0.96)

```
m, C, logev = model.predict_conditional(
    X, observed={0: y1})
```

Example 3: inverse design by gradient ascent



- Target properties $y^* = (1.8,\ 1.0)$
- Maximize $\log p(y^* \mid x)$ over x
- Tree is smooth in x, so use `jax.grad`
- Multi-start Adam finds **several** solutions
- Partial targets and box bounds supported

Pros, cons and limitations

Strengths

Uncertainty is built in

Mixture predictive with aleatoric / routing split, quantiles, samples, full covariance.

Calibrated

Post-hoc γ and α fit on held-out data; coverage tracks nominal.

Differentiable everywhere

End-to-end training, gradients in x for inverse design, composable with JAX.

Multi-output for free

Closed-form conditioning on observed outputs; correlated predictions.

Oblique splits

Fewer leaves for diagonal or curved structure than axis-aligned trees.

Familiar API

scikit-learn estimators; small codebase (JAX + optax).

Weaknesses and limitations

Fixed, full depth

$2^D - 1$ splits; practical depth ≲ 10 to 12. No grown or pruned trees yet.

Less interpretable

Oblique, soft splits are harder to read than "$x_3 < 0.42$".

Local inverse design

Multimodal likelihood; relies on multi-start ascent.

Every leaf is evaluated

Soft routing costs $O(2^D)$ per input, not $O(D)$ like a hard tree.

Non-convex training

Seed sensitive: a 1D test gave RMSE 0.16 to 0.35 at depth 4 across 4 seeds.

Single tree, aleatoric-heavy UQ

No ensemble or Bayesian treatment of parameters; linear splits only for now.

Take-home

> **difftree** = soft oblique splits + Gaussian leaves + calibration.
> One differentiable model gives a predictive **distribution**, not just a number.

- Good fit: tabular, moderate-size problems that need **calibrated UQ**
- Multi-output properties: **conditioning** and **inverse design** fall out
- Watch: depth limits, seeds, and multi-start for design

Questions?

References

L. Breiman, J. Friedman, R. Olshen, C. Stone, *Classification and Regression Trees*, Chapman & Hall/CRC (1984).

J. R. Quinlan, Induction of decision trees, *Machine Learning* 1, 81 (1986). doi:10.1007/BF00116251

P. Kontschieder et al., Deep Neural Decision Forests, *ICCV* (2015). doi:10.1109/ICCV.2015.172

N. Frosst, G. Hinton, Distilling a Neural Network Into a Soft Decision Tree, arXiv:1711.09784 (2017).

C. Guo et al., On Calibration of Modern Neural Networks, *ICML* (2017). arXiv:1706.04599

T. M. Hehn, J. F. P. Kooij, F. A. Hamprecht, End-to-End Learning of Decision Trees and Forests, *IJCV* 128, 997 (2020). doi:10.1007/s11263-019-01237-6

Figures: `presentation/make_figures.py`, `presentation/make_tree_svg.py`